PROYECTO FINAL

Inventario y ventas multisucursal

Diseno de base de datos relacional y API REST

SQL Server + FastAPI | 24 tablas relacionadas | 3FN

27 de octubre de 2026

Repositorio de entrega

https://github.com/BansheeUmg/Proyecto_Final_BD_API

Documento de analisis, reglas, modelo, dependencias funcionales, arquitectura y validacion. El codigo, los scripts y el diagrama ER completo se incluyen en el repositorio.

Proyecto final de Bases de Datos I. Fecha de elaboracion: 27 de octubre de 2026.

Repositorio: https://github.com/BansheeUmg/Proyecto_Final_BD_API

Problema y contexto

Una distribuidora simulada de tecnologia y suministros de oficina opera en Guatemala y Quetzaltenango. Sus hojas de calculo separan ventas, compras y existencias. Dos vendedores pueden ofrecer simultaneamente la ultima unidad; una recepcion puede duplicarse y los reportes pueden contar una venta varias veces al unir pagos y detalles. Se requiere una fuente relacional consistente que mantenga la trazabilidad de cada entrada y salida.

Los datos de personas, empresas, correos y telefonos son ficticios. Los importes estan expresados en quetzales, sin calculo fiscal ni conversion monetaria. Las fechas operativas se almacenan en UTC; los reportes agrupan por mes UTC.

Objetivo y alcance

Disenar e implementar una base normalizada y una API que permitan administrar catalogos, registrar ventas cobradas de contado y recibir compras sin comprometer la integridad del inventario. La solucion entrega 24 tablas, cinco CRUD, dos procedimientos atomicos, dos reportes agregados, OpenAPI y pruebas reales con SQL Server.

Se incluye informacion geografica, personal y roles, clientes, proveedores, productos, almacenes, compras, ventas, pagos y movimientos. No incluye facturacion tributaria, credito, devoluciones, transferencias entre almacenes, autenticacion ni una pantalla web. El sistema se ejecuta localmente mediante Swagger; el esquema de roles documenta responsabilidades comerciales y no implementa permisos de acceso.

Usuarios y necesidades

- Administracion: mantener catalogos, sucursales, proveedores y responsables; consultar indicadores.

- Ventas: consultar productos, registrar clientes y confirmar una venta con su pago completo.

- Bodega: recibir ordenes pendientes y consultar existencias por almacen y categoria.

- Gerencia: conocer ingresos mensuales, cantidad de ventas, ticket promedio y disponibilidad.

- Docente: reproducir la instalacion, revisar DFs y demostrar integridad y rollback.

Reglas de negocio

1. SKU, NIT de clientes y proveedores, nombres de catalogos y correo de empleado identifican registros sin duplicados dentro de su tabla (UNIQUE). 2. Los precios y costos son positivos; las cantidades de detalle son enteros positivos y las existencias nunca son negativas (CHECK y validacion Pydantic). 3. Cada producto pertenece a una categoria, una marca y una unidad existentes; cada almacen pertenece a una sucursal existente (FK obligatorias). 4. Una venta tiene al menos una linea sin productos repetidos. No puede utilizar productos inactivos ni stock insuficiente (procedimiento y Pydantic). 5. El empleado de una operacion debe estar activo y pertenecer a la sucursal del almacen (validacion en ambos procedimientos). 6. Una venta confirma simultaneamente cabecera, lineas, descuento de stock, movimientos y un pago por el total. Una

falla revierte todos esos cambios (transaccion). 7. Una referencia UUID de venta solo puede utilizarse una vez. La repeticion devuelve error y evita cobrar de nuevo (UNIQUE y bloqueo de rango). 8. Solo una compra PENDIENTE y con detalle puede recibirse. La recepcion agrega stock, genera movimientos y cambia a RECIBIDA de forma atomica. Una segunda recepcion se rechaza. 9. El movimiento pertenece a una venta o una compra, nunca a ambas: ventas con cantidad negativa, compras con cantidad positiva (CHECK). 10. No se elimina un catalogo que tenga registros dependientes; las FK no usan eliminacion en cascada. El CRUD devuelve 400 ante esta situacion. 11. Cada empleado tiene a lo sumo un perfil de contacto y fecha de ingreso (FK UNIQUE). Se permite crear primero el empleado y despues su perfil. 12. El precio de cada linea de venta se conserva como hecho historico de la transaccion; modificar el precio vigente del producto no cambia ventas anteriores.

Las reglas que cruzan tablas se imponen por procedimientos y por la ausencia de endpoints que modifiquen directamente el libro de movimientos. Un administrador con DML directo puede alterar informacion fuera de ese flujo; los permisos de produccion deben impedirlo. No se afirma que el modelo de roles implemente seguridad de API.

Modelo y justificacion de las 24 tablas

Tabla	Funcion en el dominio
Regiones	Agrupar municipios territorialmente
Municipios	Ubicar sucursales y direcciones
Sucursales	Identificar puntos de operacion
Almacenes	Separar saldos por ubicacion fisica
Roles	Catalogar responsabilidades del personal
Empleados	Identificar responsables por sucursal
PerfilesEmpleado	Mantener contacto y fecha de ingreso en una extension opcional uno a uno
EmpleadoRoles	Resolver asignaciones N:M de responsabilidades
Clientes	Identificar compradores y datos fiscales basicos
DireccionesCliente	Registrar multiples domicilios por comprador
Proveedores	Identificar abastecedores
Categorias	Clasificar productos para administracion y reportes
Marcas	Evitar repetir informacion de fabricante
Unidades	Definir la presentacion comercial del producto
Productos	Administrar SKU, descripcion y precio vigente
ProductoProveedores	Resolver N:M y guardar costo cotizado por abastecedor
Existencias	Conservar saldo y minimo por producto y almacen
Compras	Identificar orden, proveedor, almacen, empleado y estado

Tabla	Funcion en el dominio
DetalleCompra	Registrar cantidades y costos pactados por producto
Ventas	Identificar transaccion, comprador, lugar, empleado y referencia
DetalleVenta	Preservar cantidades y precios vendidos
MetodosPago	Clasificar el medio usado para cobrar
Pagos	Registrar cobros y sus importes
MovimientosInventario	Auditar cada entrada o salida con su documento de origen

Todas las tablas tienen una PK id IDENTITY. Las asociaciones conservan ademas una clave candidata compuesta UNIQUE, de modo que la PK artificial no oculta sus dependencias naturales. Las FK se indexan para consultas y comprobacion de referencias.

Cardinalidades y diagrama ER

El archivo diagrama_er.mmd incluye entidades, todos los atributos, PK, FK, claves unicas simples y cardinalidades en Mermaid; diagrama_er.md lo muestra en GitHub. El catalogo JSON conserva los CHECK, DEFAULT, nulabilidad y claves unicas compuestas que no se expresan por completo en el diagrama.

Empleado a perfil es 1:0..1, con participacion obligatoria del perfil hacia su empleado; no se fuerza un perfil al crear el empleado. Categoria a producto, cliente a venta, venta a detalle y sucursal a almacen son 1:N. Empleado a rol, producto a proveedor y producto a almacen son N:M resueltas por sus respectivas tablas asociativas. Cada movimiento tiene una FK opcional hacia venta y otra hacia compra; el CHECK exige exactamente un origen. No existe una relacion directa N:M sin resolver.

Normalizacion y dependencias funcionales

Se parte de registros operativos que mezclaban datos de productos, compradores, proveedores y lineas en una misma hoja. Las DFs describen hechos del negocio; no se infieren de coincidencias en los datos de prueba. Para todas las tablas se asume que los nombres de personas no son identificadores. El atributo id es una clave sustituta; se analizan tambien claves candidatas naturales.

Caso 1: Productos

Relacion inicial: ProductoRaw(sku, nombre, precio, activo, categoria_id, categoria_nombre, marca_id, marca_nombre, unidad_id, unidad_nombre, lista_proveedores). DFs: sku -> nombre, precio, activo, categoria_id, marca_id, unidad_id; categoria_id -> categoria_nombre; marca_id -> marca_nombre; unidad_id -> unidad_nombre.

1FN: se elimina lista_proveedores y se crea una fila por par producto/proveedor en ProductoProveedores. Cada producto conserva un solo valor de categoria, marca y unidad; no se almacenan listas ni columnas proveedor1/proveedor2.

2FN: en Productos la clave natural sku es simple y no tiene subconjuntos propios, por lo que no existen dependencias parciales. En la asociacion (producto_id, proveedor_id) -> costo; el costo depende del par, porque un mismo producto puede tener precios de compra distintos por proveedor.

3FN: se separan Categorias(categoria_id, nombre), Marcas(marca_id, nombre) y Unidades(unidad_id, nombre), eliminando sku -> categoria_id -> categoria_nombre y las transitividades equivalentes. Relacion final Productos(id, sku, nombre, precio, activo, categoria_id, marca_id, unidad_id). Claves candidatas: {id} y {sku}; ambas determinan todos sus atributos. Los nombres de los catalogos no permanecen en Productos. Todo determinante no trivial declarado en Productos es superclave; cumple 3FN (y BCNF bajo estas DFs).

Caso 2: DetalleVenta

Relacion inicial: VentaLineaRaw(venta_id, producto_id, fecha, cliente_id, cliente_nombre, producto_nombre, cantidad, precio_unitario). Clave candidata: (venta_id, producto_id), asumiendo una sola linea por producto en cada venta. DFs: venta_id -> fecha, cliente_id; cliente_id -> cliente_nombre; producto_id -> producto_nombre; (venta_id, producto_id) -> cantidad, precio_unitario.

1FN: una fila representa un producto vendido, con cantidad y precio atomicos. Las lineas repetidas del mismo producto se rechazan.

2FN: fecha y cliente_id dependen solo de venta_id; producto_nombre depende solo de producto_id. Se trasladan la cabecera a Ventas y la descripcion a Productos para eliminar dependencias parciales de la clave compuesta.

3FN: cliente_nombre se ubica en Clientes para eliminar venta_id -> cliente_id -> cliente_nombre. Relacion final DetalleVenta(id, venta_id, producto_id, cantidad, precio_unitario), con claves {id} y {venta_id, producto_id}. DFs: cada clave -> todos los demas atributos. El precio_unitario es el precio aplicado a esa venta: producto_id no lo determina, pues el precio puede cambiar en el tiempo. Cada determinante no trivial es superclave; cumple 3FN. El total no se almacena en la cabecera: se calcula como SUM(cantidad * precio_unitario).

Caso 3: Existencias

Relacion inicial: StockRaw(almacen_id, producto_id, almacen_nombre, sucursal_id, sucursal_nombre, producto_nombre, cantidad, minimo). Clave candidata: (almacen_id, producto_id). DFs: almacen_id -> almacen_nombre, sucursal_id; sucursal_id -> sucursal_nombre; producto_id -> producto_nombre; (almacen_id, producto_id) -> cantidad, minimo.

1FN: cada fila contiene el saldo de un producto en un almacen; se eliminan columnas AlmacenCentral/AlmacenOccidente. Cantidad y minimo son enteros atomicos.

2FN: almacen_nombre y sucursal_id dependen de parte de la clave compuesta, al igual que producto_nombre. Se separan en Almacenes y Productos, dejando solo los hechos dependientes del par.

3FN: sucursal_nombre reside en Sucursales para eliminar almacen_id -> sucursal_id -> sucursal_nombre. Relacion final Existencias(id, almacen_id, producto_id, cantidad, minimo), con claves {id} y {almacen_id, producto_id}. El minimo es una politica por ubicacion y producto, no un atributo global del producto. Ambas claves determinan cantidad y minimo y no hay dependencias entre atributos no clave; cumple 3FN.

Revision global

Los catalogos tienen claves simples; las tablas asociativas tienen UNIQUE sobre sus pares naturales. En Municipios la clave candidata adicional es (region_id, nombre); en Almacenes, (sucursal_id, nombre). En PerfilesEmpleado tanto id como empleado_id son claves candidatas. Las cabeceras guardan hechos del evento y referencias, sin nombres copiados de otras entidades. Pagos guarda cada cobro como hecho independiente; una venta de contado creada por el procedimiento genera uno solo. El esquema permite

multiples cobros para una futura ampliacion, pero no ofrece un endpoint para fraccionarlos.

Existencias es una proyeccion operativa actualizada junto con el libro de movimientos. Esta duplicacion controlada entre saldo y eventos reduce el costo de validar stock y exige mantener ambos en una misma transaccion. La semilla incluye saldos iniciales anteriores al periodo de movimientos; por ello el saldo no debe interpretarse como la suma exclusiva de los movimientos de demostracion. La 3FN se analiza por relacion y no implica eliminar un libro de eventos ni precios historicos.

Operaciones atomicas y concurrencia

RegistrarVenta valida el arreglo JSON, duplicados, cliente, metodo de pago, empleado/sucursal, productos y saldo. Obtiene UPDLOCK y HOLDLOCK sobre la referencia y las existencias; conserva los bloqueos hasta finalizar la transaccion. Inserta venta y lineas, descuenta saldos, inserta movimientos y pago, y confirma. Una segunda solicitud que compita por la misma unidad debe esperar y verificar el saldo resultante.

RecibirCompra bloquea la cabecera y valida estado PENDIENTE, responsable y detalle. Crea saldos inexistentes con proteccion del rango, incrementa cantidades, registra movimientos y cambia el estado. El bloqueo de cabecera impide recibir dos veces la misma compra. Ambos procedimientos usan SET XACT_ABORT ON, BEGIN TRANSACTION, TRY/CATCH, COMMIT y ROLLBACK condicionado por XACT_STATE; THROW preserva el error.

La API utiliza conexiones autocommit para que la transaccion del procedimiento sea la unidad de trabajo. Cada solicitud obtiene su propia conexion y la cierra al terminar. Cada operacion CRUD consta de una instruccion DML atomica con OUTPUT, evitando insertar y luego consultar con otra conexion. No se garantiza ausencia total de deadlocks en cargas arbitrarias; un conflicto del motor devuelve error controlado sin saldo negativo.

Reportes y preguntas de negocio

Ingresos mensuales por sucursal: cuantas ventas hubo y cual fue el ingreso, ticket promedio, venta minima y maxima. Se totaliza primero cada venta con SUM en una CTE y despues se agrupa por sucursal, anio y mes. COUNT, SUM, AVG, MIN, MAX y HAVING se aplican al nivel correcto. No se unen pagos al detalle, evitando multiplicar importes.

Inventario por categoria: cuantos productos y unidades existen y cual es la disponibilidad media, minima y maxima por producto. La CTE suma todas las ubicaciones de cada producto e incluye saldo cero cuando no existe fila de existencias. Luego agrega por categoria; HAVING filtra unidades minimas. COUNT cuenta productos distintos por construccion de la CTE, sin inflarse por los almacenes.

Arquitectura y contrato HTTP

app/database.py gestiona variables y conexiones; app/schemas.py define entradas y respuestas; app/services/crud.py concentra persistencia de catalogos; app/routers separa catalogos, transacciones y reportes. app/main.py compone routers y traduce errores a JSON. Los identificadores SQL se toman de un registro interno fijo; todos los valores externos se parametrizan con ?. No hay credenciales en el repositorio.

Los cinco recursos CRUD son categorias, marcas, clientes, proveedores y productos. Cada uno ofrece POST, GET listado paginado, GET por ID, PUT completo y DELETE. En total hay 25 operaciones CRUD, 2 operaciones transaccionales, 2 reportes y 1 endpoint de estado: 30 operaciones, distribuidas en 15 rutas OpenAPI. Swagger documenta esquemas, parametros y codigos 200/201/204/400/404/500 segun

corresponda. La validacion se personaliza a 400 y no se anuncia 422.

Verificacion y demostracion

La suite de integracion utiliza una base exclusiva InventarioFinalTests y valida su nombre antes de modificarla. Cubre CRUD de las cinco tablas, FK, UNIQUE, borrado restringido, entradas invalidas, referencia repetida, venta exitosa, recepcion y rechazo de reintentos, agregados y Swagger. Ademas inyecta un trigger de prueba que falla al insertar el pago, despues de crear venta, detalles, movimientos y descontar stock: el rollback elimina todos los cambios, incluido el trigger creado dentro de la transaccion externa de prueba.

Una prueba concurrente usa dos conexiones reales sobre un producto temporal con una unidad; solo una confirma y la otra recibe el error de stock. Se eliminan los registros de prueba asociados. La evidencia y resultados ejecutados se registran en validacion.md. El README detalla comandos de instalacion, SQL, ejecucion y pruebas, ademas de una venta de ejemplo.

Referencias

- Requisitos academicos: documento Proyecto Final proporcionado por el usuario.

- FastAPI, Bigger Applications - Multiple Files: https://fastapi.tiangolo.com/tutorial/bigger-applications/

- Microsoft Learn, TRY...CATCH: https://learn.microsoft.com/en-us/sql/t-sql/language-elements/try-catch-transact-sql

- Microsoft Learn, SET XACT_ABORT: https://learn.microsoft.com/en-us/sql/t-sql/statements/set-xact-abort-transact-sql

- Microsoft Learn, Transaction Locking and Row Versioning Guide: https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide

Las fuentes oficiales se consultaron para la organizacion modular y el comportamiento de transacciones. El dominio, las DFs y el modelo son decisiones de este proyecto.

Anexo A. Diccionario de datos

PK: id INT IDENTITY en cada tabla. Las FK son obligatorias salvo venta_id y compra_id de MovimientosInventario. Los atributos y las restricciones se reproducen del catalogo fuente; el script SQL es la definicion ejecutable.

Regiones

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(80) NOT NULL UNIQUE

Municipios

id INT IDENTITY PRIMARY KEY; region_id INT NOT NULL REFERENCES Regiones(id), nombre NVARCHAR(80) NOT NULL; UNIQUE(region_id,nombre)

Sucursales

id INT IDENTITY PRIMARY KEY; municipio_id INT NOT NULL REFERENCES Municipios(id), nombre NVARCHAR(100) NOT NULL UNIQUE, direccion NVARCHAR(200) NOT NULL

Almacenes

id INT IDENTITY PRIMARY KEY; sucursal_id INT NOT NULL REFERENCES Sucursales(id), nombre NVARCHAR(80) NOT NULL; UNIQUE(sucursal_id,nombre)

Roles

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(80) NOT NULL UNIQUE

Empleados

id INT IDENTITY PRIMARY KEY; sucursal_id INT NOT NULL REFERENCES Sucursales(id), nombre NVARCHAR(100) NOT NULL, email NVARCHAR(160) NOT NULL UNIQUE, activo BIT NOT NULL DEFAULT 1

PerfilesEmpleado

id INT IDENTITY PRIMARY KEY; empleado_id INT NOT NULL UNIQUE REFERENCES Empleados(id), telefono NVARCHAR(20) NOT NULL, fecha_ingreso DATE NOT NULL

EmpleadoRoles

id INT IDENTITY PRIMARY KEY; empleado_id INT NOT NULL REFERENCES Empleados(id), rol_id INT NOT NULL REFERENCES Roles(id); UNIQUE(empleado_id,rol_id)

Clientes

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(100) NOT NULL, nit NVARCHAR(20) NOT NULL UNIQUE, email NVARCHAR(160) NOT NULL

DireccionesCliente

id INT IDENTITY PRIMARY KEY; cliente_id INT NOT NULL REFERENCES Clientes(id), municipio_id INT NOT NULL REFERENCES Municipios(id), direccion NVARCHAR(200) NOT NULL

Proveedores

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(100) NOT NULL, nit NVARCHAR(20) NOT NULL UNIQUE, email NVARCHAR(160) NOT NULL

Categorias

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(80) NOT NULL UNIQUE

Marcas

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(80) NOT NULL UNIQUE

Unidades

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(40) NOT NULL UNIQUE

Productos

id INT IDENTITY PRIMARY KEY; categoria_id INT NOT NULL REFERENCES Categorias(id), marca_id INT NOT NULL REFERENCES Marcas(id), unidad_id INT NOT NULL REFERENCES Unidades(id), sku NVARCHAR(40) NOT NULL UNIQUE, nombre NVARCHAR(100) NOT NULL, precio DECIMAL(12,2) NOT NULL CHECK(precio>0), activo BIT NOT NULL DEFAULT 1

ProductoProveedores

id INT IDENTITY PRIMARY KEY; producto_id INT NOT NULL REFERENCES Productos(id), proveedor_id INT NOT NULL REFERENCES Proveedores(id), costo DECIMAL(12,2) NOT NULL CHECK(costo>0); UNIQUE(producto_id,proveedor_id)

Existencias

id INT IDENTITY PRIMARY KEY; almacen_id INT NOT NULL REFERENCES Almacenes(id), producto_id INT NOT NULL REFERENCES Productos(id), cantidad INT NOT NULL DEFAULT 0 CHECK(cantidad>=0), minimo INT NOT NULL DEFAULT 5 CHECK(minimo>=0); UNIQUE(almacen_id,producto_id)

Compras

id INT IDENTITY PRIMARY KEY; proveedor_id INT NOT NULL REFERENCES Proveedores(id), almacen_id INT NOT NULL REFERENCES Almacenes(id), empleado_id INT NOT NULL REFERENCES Empleados(id), fecha DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), estado NVARCHAR(12) NOT NULL DEFAULT 'PENDIENTE' CHECK(estado IN ('PENDIENTE','RECIBIDA'))

DetalleCompra

id INT IDENTITY PRIMARY KEY; compra_id INT NOT NULL REFERENCES Compras(id), producto_id INT NOT NULL REFERENCES Productos(id), cantidad INT NOT NULL CHECK(cantidad>0), costo DECIMAL(12,2) NOT NULL CHECK(costo>0); UNIQUE(compra_id,producto_id)

Ventas

id INT IDENTITY PRIMARY KEY; cliente_id INT NOT NULL REFERENCES Clientes(id), almacen_id INT NOT NULL REFERENCES Almacenes(id), empleado_id INT NOT NULL REFERENCES Empleados(id), fecha DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(), referencia UNIQUEIDENTIFIER NOT NULL UNIQUE

DetalleVenta

id INT IDENTITY PRIMARY KEY; venta_id INT NOT NULL REFERENCES Ventas(id), producto_id INT NOT NULL REFERENCES Productos(id), cantidad INT NOT NULL CHECK(cantidad>0), precio_unitario DECIMAL(12,2) NOT NULL CHECK(precio_unitario>0); UNIQUE(venta_id,producto_id)

MetodosPago

id INT IDENTITY PRIMARY KEY; nombre NVARCHAR(60) NOT NULL UNIQUE

Pagos

id INT IDENTITY PRIMARY KEY; venta_id INT NOT NULL REFERENCES Ventas(id), metodo_pago_id INT NOT NULL REFERENCES MetodosPago(id), monto DECIMAL(14,2) NOT NULL CHECK(monto>0), fecha DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()

MovimientosInventario

id INT IDENTITY PRIMARY KEY; almacen_id INT NOT NULL REFERENCES Almacenes(id), producto_id INT NOT NULL REFERENCES Productos(id), venta_id INT NULL REFERENCES Ventas(id), compra_id INT NULL REFERENCES Compras(id), cantidad INT NOT NULL CHECK(cantidad<>0), fecha DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(); CHECK((venta_id IS NOT NULL AND compra_id IS NULL AND cantidad<0) OR (venta_id IS NULL AND compra_id IS NOT NULL AND cantidad>0))

Anexo B. Relaciones del modelo ER

El archivo Mermaid contiene la representacion grafica con todos los atributos. Esta matriz proporciona una vista legible de cada relacion. 1:N permite cero o mas hijos; 1:0..1 es una extension opcional.

Padre	Hijo / FK	Cardinalidad
Regiones	Municipios.region_id	1:N
Municipios	Sucursales.municipio_id	1:N
Sucursales	Almacenes.sucursal_id	1:N
Sucursales	Empleados.sucursal_id	1:N
Empleados	PerfilesEmpleado.empleado_id	1:0..1
Empleados	EmpleadoRoles.empleado_id	1:N
Roles	EmpleadoRoles.rol_id	1:N
Clientes	DireccionesCliente.cliente_id	1:N
Municipios	DireccionesCliente.municipio_id	1:N
Categorias	Productos.categoria_id	1:N
Marcas	Productos.marca_id	1:N
Unidades	Productos.unidad_id	1:N
Productos	ProductoProveedores.producto_id	1:N
Proveedores	ProductoProveedores.proveedor_id	1:N
Almacenes	Existencias.almacen_id	1:N
Productos	Existencias.producto_id	1:N
Proveedores	Compras.proveedor_id	1:N
Almacenes	Compras.almacen_id	1:N
Empleados	Compras.empleado_id	1:N
Compras	DetalleCompra.compra_id	1:N
Productos	DetalleCompra.producto_id	1:N
Clientes	Ventas.cliente_id	1:N
Almacenes	Ventas.almacen_id	1:N
Empleados	Ventas.empleado_id	1:N
Ventas	DetalleVenta.venta_id	1:N
Productos	DetalleVenta.producto_id	1:N
Ventas	Pagos.venta_id	1:N
MetodosPago	Pagos.metodo_pago_id	1:N
Almacenes	MovimientosInventario.almacen_id	1:N
Productos	MovimientosInventario.producto_id	1:N

Padre	Hijo / FK	Cardinalidad
Ventas	MovimientosInventario.venta_id	1:N
Compras	MovimientosInventario.compra_id	1:N

La tabla MovimientosInventario restringe con CHECK el origen exclusivo: venta o compra. Las asociaciones EmpleadoRoles, ProductoProveedores y Existencias resuelven N:M mediante dos relaciones 1:N y una clave compuesta UNIQUE.

Anexo C. Evidencia de validacion

Fecha local: 27 de octubre de 2026. Entorno: Windows, Python 3.12, SQL Server SQLEXPRESS y ODBC Driver 17. Autenticacion integrada de Windows.

El script schema_and_data.sql se ejecuto satisfactoriamente con sqlcmd -b. Se instalaron la base InventarioFinal y una base independiente InventarioFinalTests con los mismos objetos. El modelo final contiene 24 tablas y dos procedimientos. Todas las tablas tienen datos iniciales; los datos de personas y empresas son ficticios.

Resultado de pytest: 15 passed. Se ejecutaron cinco ciclos CRUD completos, restricciones, validaciones, venta confirmada, recepcion confirmada, rechazo de reintentos y reportes con contraste de totales.

Rollback temprano: una venta con dos productos y stock insuficiente en una linea no altero cabecera, detalles, pagos, movimientos ni el saldo total.

Rollback tardio: un trigger temporal rechazo el pago despues del descuento de stock. La transaccion revirtio todos los cambios; se comprobaron conteos y saldos iguales al estado anterior.

Concurrencia: dos conexiones independientes intentaron vender la unica unidad de un producto temporal. Se confirmo una venta y se rechazo la otra por stock insuficiente. El saldo final fue cero y los registros temporales fueron eliminados.

Reportes sobre datos iniciales: ingresos totales Q 9,420.00, dos ventas; se verifico que el reporte de inventario no multiplica el numero de productos por los almacenes. El umbral de ingresos excluye grupos por HAVING.

Swagger /docs y /openapi.json devolvieron 200 durante las pruebas. Se verificaron 15 rutas y la respuesta personalizada 400 para validaciones. La suite usa conexiones reales a SQL Server, sin sustituir el motor por SQLite ni simular resultados SQL.

Se observo un aviso de deprecacion de la biblioteca de pruebas sobre httpx; no afecto las 15 comprobaciones. Las pruebas no se ejecutan si falta TEST_ODBC_CONNECTION_STRING y muestran explicitamente skipped en ese caso.

El esquema de roles no implementa autenticacion. La API se inicia en 127.0.0.1 para la demostracion local. No se realizo una publicacion del servicio en Internet.